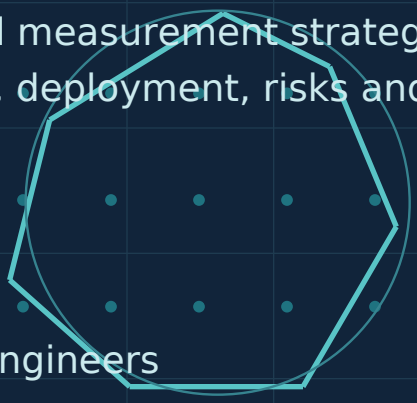
HACKATHON / BUILD SPECIFICATION V1.0

GHOSTGRIP

AI-Powered Adaptive Virtual Grasping
with Physical Haptic Feedback

Complete, end-to-end technical documentation: hardware inventory, AI pipeline, simulation design, haptic actuator and measurement strategy, software stack, interfaces, safety, experiments, deployment, risks and a 36-hour implementation roadmap.



Build duration: 36 hours **Target team:** 3-4 engineers

Available: Arduino Uno + servo motor + Seeed Grove starter kit

Last updated: November 7, 2026

DESIGN STATUS: Engineering proposal, not a finished or validated system.

DOCUMENT GUIDE

Contents

This document is designed for a hackathon team to implement and present a working prototype from first principles. The acceptance targets are engineering goals; all final performance numbers must be measured during the event.

Contents . . . 2

1. Executive summary and product definition . . . 4

2. Hardware: confirmed inventory, requirements and gaps . . . 5

3. System architecture and control loop . . . 7

4. AI design: perception, intention and movement prediction . . . 10

5. Simulation: virtual hand, objects and grasp physics . . . 12

6. Haptic mechanism: what it can and cannot do . . . 14

7. Pin mapping, circuit plan and Arduino firmware . . . 16

8. Software stack and repository structure . . . 18

9. End-to-end interactions and user experience . . . 21

10. Logging, measurements and evaluation strategy . . . 22

11. Implementation plan: 36-hour hackathon . . . 24

12. Testing strategy and fault tolerance . . . 25

13. Demonstration, judging story and technical evidence . . . 27

14. Deployment, operations, privacy and reproducibility . . . 28

15. Decision log and open engineering questions . . . 29

16. Implementation pseudocode and integration contract . . . 30

17. Suggested enhancements only after MVP passes . . . 31

18. Reference documentation and evidence basis 31

19. Final team launch checklist 33

Read this before connecting hardware

Do not attach the servo to a human finger until its model, power requirements, travel limits and mechanical clearance are confirmed. Start with non-contact motion and use soft, low-force cues only if safety is demonstrated. The included plug power adapter is **not automatically a suitable direct servo supply**.

1. Executive summary and product definition

GhostGrip is a laptop-based adaptive interaction prototype. A webcam estimates hand landmarks and grasp-related motion; a lightweight prediction and personalization stack assists interaction with simple simulated objects; an Arduino translates safe virtual events into physical cues using a servo and Grove outputs. The project is a **human-computer interaction and control-system demonstration**, not a diagnostic, therapeutic or clinically validated rehabilitation device.

1.1 Problem, intended users and proposed value

Fine hand-control tasks can be difficult to practice, evaluate or adapt with simple button-driven software. Many virtual interfaces provide visual feedback but little physical correspondence. GhostGrip explores whether a personalized virtual grasp task, optionally paired with a physical cue, makes interaction clearer or more successful for a user.

- **Primary use case:** experimental adaptive grasp practice with healthy volunteer testers during a hackathon.
- **Long-term potential:** research on accessible interfaces, procedural training, hand tracking and teleoperation - subject to substantial future validation.
- **Core differentiator:** perception, temporal prediction, adaptive assistance, controllable virtual contact and instrumented physical feedback in one end-to-end loop.

1.2 Product requirements and non-goals

ID	Requirement	Observable acceptance criterion
FR-01	Track one hand through a webcam	Live landmarks and confidence/loss state update in the simulator.
FR-02	Recognize approach, close, hold and release	Grasp state transitions are visible; false triggers can be counted.
FR-03	Simulate at least three object types	Soft, fragile and heavy virtual objects react differently.
FR-04	Adapt to user-specific movement	Calibration changes thresholds; baseline and adaptive modes are switchable.
FR-05	Generate Arduino feedback	At least one safe servo command is caused by a virtual contact event.
FR-06	Log metrics and run experiments	Session CSV includes condition, outcomes, confidence, cue times and sensor data.
NFR-01	Graceful failure	Camera loss, serial loss or invalid commands disable physical output.
NFR-02	Reproducible demo	Can run a laptop-only fallback and replay a recorded synthetic trace.

Explicit non-goals

No measurement of real grip force, no realistic whole-hand force feedback, no clinical claims, no adaptive deep reinforcement-learning training, no general 3D biomechanical hand model, no requirement to identify an individual user.

2. Hardware: confirmed inventory, requirements and gaps

2.1 Confirmed separately from the Grove kit

Item	Status	Proposed role	Verification
Arduino Uno	CONFIRMED / in hand	USB serial peripheral for sensors, outputs and servo.	Confirm Uno revision and USB communication.
Servo motor	CONFIRMED / in hand	Moves small padded indicator or low-force tactile contact arm.	Exact model, rated voltage, current and travel limits UNKNOWN.
Laptop with webcam, microphone and USB	ASSUMED / not shown in hardware photo	All AI inference, simulation and dashboard.	Verify webcam, Windows/macOS/Linux, available USB port.
Mounting supplies: tape, cardboard, foam, brackets	NEEDED / not verified	Holds servo and sensors safely.	Borrow/build in lab; avoid unsafe pinch mechanism.

2.2 Complete Grove kit contents from the photographed box

The list below transcribes the Grove kit photograph supplied by the team. It establishes that these are listed as included in the kit, not that each individual part has been function-tested.

Kit component	Qty	GhostGrip use
Grove Base Shield V2	1	Used - connects Grove modules to Arduino Uno
Grove Buzzer	1	Optional - contact/error cue
Grove Button	1	Useful - calibration / emergency stop input
Grove Green LED	1	Used - stable grasp indication
Grove Blue LED	1	Optional - tracking/calibration status
Grove Red LED	1	Used - fault / invalid grip indication
Grove Sound Sensor	1	Not needed for MVP
Grove 3-Axis Digital Accelerometer (+/-1.5g)	1	Used in instrumented feedback assembly if mounting allows

Kit component	Qty	GhostGrip use
Grove Touch Sensor	1	Used - fingertip contact/arming switch
Grove Light Sensor	1	Not needed for MVP
Grove Temperature Sensor	1	Not needed for MVP
Grove LCD RGB Backlight	1	Optional - status/mode screen
Grove Rotary Angle Sensor(P)	1	Useful - feedback intensity / assistance knob
Grove Piezo Vibration Sensor	1	Optional - binary vibration detection
Gear Stepper Motor with Driver	1	Out of scope for MVP
Plug Power Adapter	1	Do not connect to servo until output rating verified
USB Cable	2	Used - board and spare cable
26AWG Grove Cable	14	Used - modular sensors/indicators

2.3 Bill of materials by priority

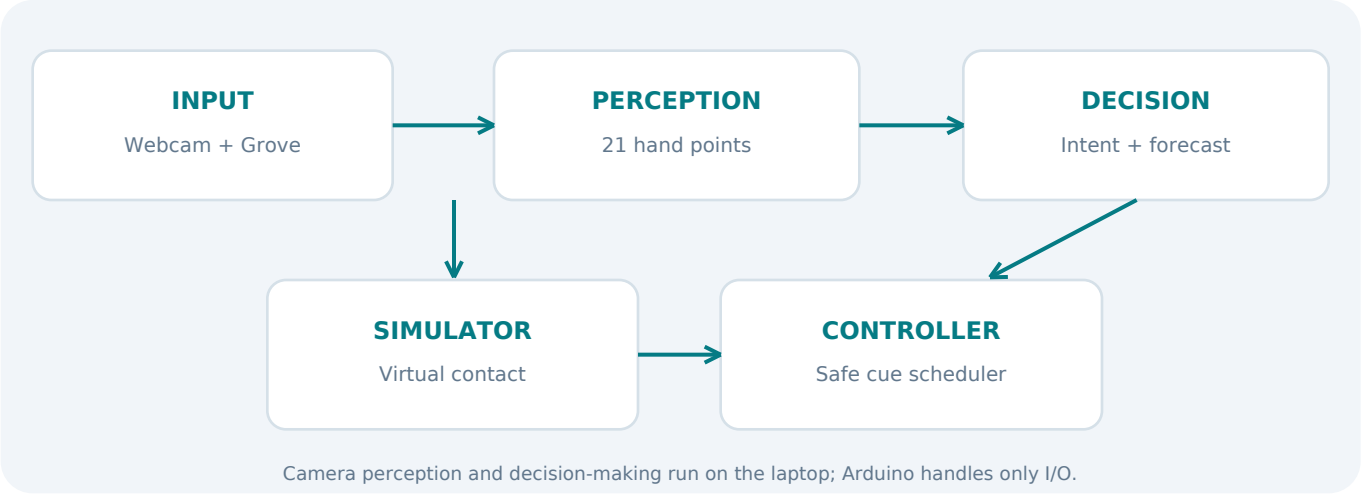
Priority	Requirement	Status / safe alternative
P0	Arduino Uno, Grove Base Shield, USB cable, laptop webcam	Confirmed except laptop is assumed available.
P0	Servo motor plus suitable mounting and control lead	Servo confirmed; model, harness and fixture unverified.
P0	Safe, model-compatible power for servo	Potential gap: verify external regulated supply requirement.
P0	MediaPipe hand model asset + Python environment	Software download/installation required.
P1	Touch sensor, button, LED and Grove leads	Listed in kit; test each.
P1	Accelerometer and piezo vibration sensor	Listed in kit; instrument only if practical.
P2	LCD RGB backlight, rotary knob and buzzer	Listed in kit; optional polish.
OPTIONAL	Force-sensing resistor or load cell with amplifier	Not in photographed kit. Required for real contact-force measurement.
OPTIONAL	Vibration motor + driver + proper supply	Not in photographed kit. Only needed for genuine vibration-actuator cues.

What is NOT confirmed

A force sensor, vibration motor, EEG headset, haptic glove, 3D printer, motor driver board, safe separate regulated servo supply, breadboard, USB port access and any particular servo model are NOT confirmed. The plan does not rely on these being available.

3. System architecture and control loop

GhostGrip splits into two independent execution domains: the laptop owns vision, predictive AI, the virtual world and experiment logging; Arduino owns physical I/O and safety interlocks. A simple serial protocol links them.



Camera perception and decision-making run on the laptop; Arduino handles only I/O.

3.1 Data pipeline

- **Capture:** webcam frame at approximately 30 fps if supported; timestamp with a monotonic host clock.
- **Perception:** MediaPipe Hand Landmarker returns 21 points and a tracking/loss state.
- **Features:** normalize fingertip distance by palm scale; estimate reach position, closing velocity, stability and smoothing.
- **Prediction:** forecast near-future hand position; classify grasp phase and compute uncertainty.
- **Simulation:** apply a fixed-time-step contact model to three virtual object types.
- **Policy:** decide whether to issue an assistance suggestion and a bounded physical cue.
- **Actuation:** send event to Arduino; read back acknowledgment and optional sensors.
- **Evaluation:** log trial events, prediction errors, completion outcomes, response times and failures.

3.2 Timing budgets and boundaries

These are **provisional engineering targets** to profile, not guaranteed performance: camera 20-30 fps; simulation 60 Hz on a typical laptop; vision inference ideally less than 45 ms per processed frame; intent updates at 10-30 Hz; serial commands at event boundaries rather than per video frame; servo cues limited in frequency and travel for safety.

3.3 Three operating loops

Loop	Suggested cadence	Why separate it
Perception and AI	Each processed webcam frame; timestamps, no fixed fps assumption	Compute-intensive and may skip frames under load.
Virtual physics	Fixed 1/60-second simulation step, with capped accumulated dt	Makes contact results less dependent on webcam fps.
Hardware and logging	Event-driven messages; sensor poll 20-50 Hz if reliable	Avoid servo chatter and keep output responsive without saturating serial.

Critical architecture invariant

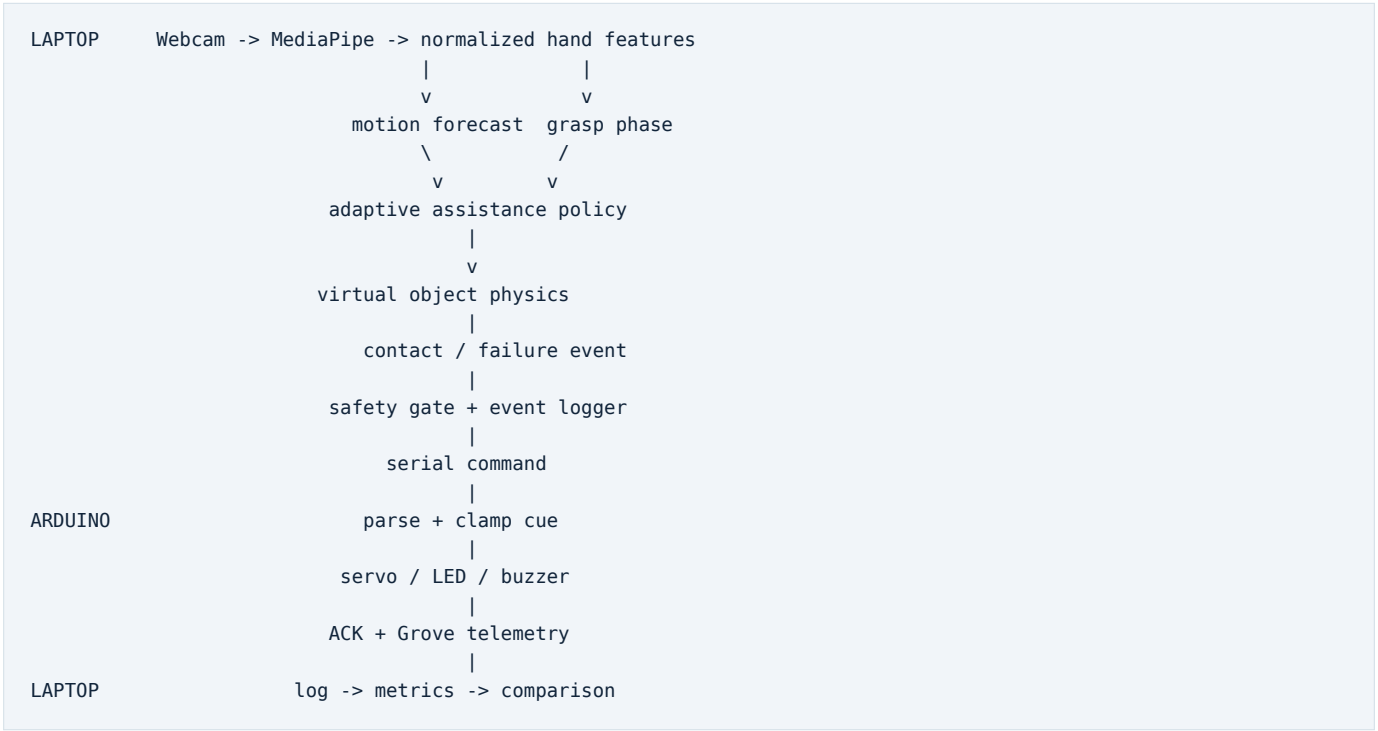
The physical servo must never be directly driven by raw camera landmarks. Every actuator command passes through a contact-state validator, rate limiter, angle clamp, confidence gate, arming state and timeout. The simulation continues safely if Arduino disconnects.

3.4 End-to-end workflow: from hand motion to physical cue

This is the **single grasp lifecycle**. Every step has an input, output, owning subsystem, and failure behavior. The servo is never driven directly by a camera measurement.

Step / owner	Processing and decision	Output / failure path
01 User + webcam	Calibrate open/closed pinch; camera timestamps each frame.	Frame(t); missing -> TRACKING_LOST.
02 Perception / laptop	MediaPipe extracts 21 landmarks; reject invalid geometry.	LandmarkFrame; failure -> neutral hand.
03 Feature extractor	Normalize thumb-index distance; smooth position and compute velocity.	HandFeatures: pinch, speed, confidence.
04 AI predictor	Forecast motion; infer grasp phase and uncertainty.	Prediction; high uncertainty disables assistance.
05 Adaptive policy	Use calibration and recent trials for bounded personalization.	AssistancePolicy with active mode flags.
06 Physics simulation	At 60 Hz, test object overlap, contact, hold, slip and over-grip.	SimEvent: CONTACT, HOLD, SLIP, RELEASE, DAMAGE.
07 Safety controller	Debounce events; enforce arming, limits, confidence, timeout.	CueEvent or SUPPRESSED(reason).
08 Arduino / outputs	Parse serial message; bounded servo and status LED; return ACK.	Physical cue or safe idle.
09 Grove feedback	Sample touch, 3-axis acceleration, optional binary piezo.	DeviceState; NOT measured grip force.
10 Logger / evaluator	Join events by timestamp; calculate outcomes and latencies.	CSV log; 4-condition comparison.

3.5 Workflow diagram: laptop intelligence and Arduino hardware



Concrete judge demonstration

The judge pinches toward a virtual fragile object. Landmarks become normalized position and closing-speed features; the predictor estimates the next motion. The simulator decides whether contact and a synthetic over-grip occur. A validated CONTACT or ERROR event can move the servo indicator once. Arduino acknowledges the command, and the optional Grove sensors record motion/vibration events. The dashboard displays the virtual result, physical cue status and measured latency.

3.6 Workflow failure gates

Failure	System response
Tracking lost / uncertainty too high	Release or freeze virtual grasp; suppress physical command.
Unexpected hand-coordinate jump	Reject feature outlier; do not infer force or intention.
Serial or ACK timeout	Continue laptop-only demo; show disconnect clearly.
Unsafe servo behavior	Disconnect actuator power; maintain virtual mode.
Missing Grove telemetry	Mark absent; do not fabricate measurements.
Stop command or watchdog event	Return to safe idle and require re-arming.

4. AI design: perception, intention and movement prediction

4.1 Webcam setup and hand landmark extraction

Use Google MediaPipe Tasks Hand Landmarker in VIDEO mode for the simplest deterministic MVP; use LIVE_STREAM mode only after the application is stable. VIDEO mode requires monotonically increasing frame timestamps; LIVE_STREAM uses a callback and may drop input frames under load [R1]. Convert OpenCV BGR images to RGB before model inference. Maintain a tracking-lost state if there is no valid hand.

- **Source input:** one webcam, one hand, uncluttered background and stable lighting.
- **Output:** 21 image landmarks (0 wrist; 4 thumb tip; 8 index tip; 5 index MCP; 17 pinky MCP, etc.) plus handedness information.
- **Calibration:** user performs open-hand, pinch, reach and release gestures for 10-20 seconds.
- **Preprocessing:** mirror the view for intuitive interaction, then consistently transform pointer coordinates; smooth noisy landmarks and reject outliers.
- **Tracking safeguards:** if hand leaves the frame, freeze or safely release the virtual object and send no new active servo command.

4.2 Features: geometry and temporal motion

Feature	Definition / estimation	Interpretation
Normalized pinch gap	d = ||thumb_tip - index_tip|| / max(||index_mcp - pinky_mcp||, epsilon)	Size-normalized opening; not a physical force.
Grip closure q	clip((d_open - d)/(d_open - d_closed), 0, 1)	0 = calibrated open, 1 = calibrated closed.
Pointer p	Midpoint of thumb tip and index tip in mapped simulator coordinates	Approximate virtual grasp location.
Velocity v	Smoothed (p[t] - p[t-dt])/dt; optionally Kalman-estimated	Approaching, retreating and movement speed.
Closure velocity	Smoothed (q[t] - q[t-dt])/dt	How rapidly fingers close/open.
Stability	Rolling median absolute deviation or std. of pointer and q	High jitter suggests unstable tracking or movement.
Confidence quality	Landmark presence + temporal consistency + in-bounds checks	Gates both assistance and physical feedback.

Use palm-scale normalization and per-user range calibration instead of treating pixel distances as meaningful across different hand sizes. A partially hidden finger can produce misleading landmarks; track quality over time, not only a single-frame detection score.

4.3 Grasp-state classification: achievable baseline before ML

State machine: **IDLE -> APPROACHING -> CLOSING -> CONTACT_CANDIDATE -> HOLDING -> RELEASING -> IDLE**. Guard transitions with location, q, closure velocity and persistence across several frames. Add a cooldown to avoid rapid re-triggering on jitter.

Transition	Illustrative rule	Failure guard
Idle -> approaching	Pointer enters expanded object boundary; velocity points toward it	Ignore low-confidence or out-of-frame detections.
Approaching -> closing	q increases for >=3 valid frames	Do not infer grasp solely from a tiny pinch distance.
Closing -> contact	Pointer overlaps grasp zone and q crosses object contact threshold	Require contiguous frames / hysteresis.
Contact -> holding	Virtual force surrogate exceeds required simulated hold threshold	Object must be graspable and within allowed position.
Holding -> releasing	q decreases below release threshold for several frames	Hysteresis prevents release chatter.

Optional trainable enhancement: collect 60-120 short, manually tagged action windows during the hackathon; train a small logistic regression / random forest over engineered features. Use leave-one-session-out evaluation where possible, keep the rule-based state machine as the fallback, and report the limited sample size. A classifier output is not proof of human intention.

4.4 Motion prediction

Start with a constant-velocity state estimator for the pointer: state x = [px, py, vx, vy]. Predict x(t + h) using the state transition matrix with horizon h = 0.10-0.20 s. Correct with new observed positions. A simple smoothed-velocity extrapolation is the baseline. Clamp predicted positions to the simulation boundary and decrease assistance as prediction uncertainty grows.

```
x = [px, py, vx, vy]
F(dt) = [[1,0,dt,0], [0,1,0,dt], [0,0,1,0], [0,0,0,1]]
x_pred = F(dt) @ x_est
x_est = kalman_update(x_pred, measured_pointer, R)
p_future = x_est[:2] + horizon_seconds * x_est[2:4]
```

Track mean absolute error (MAE) of predicted location after the actual horizon has elapsed. Use a ring buffer of timestamped predictions; align each with the subsequent observed location, not the same frame. Compare to a no-movement and constant-velocity baseline. If error is worse, reduce or disable predictive assistance.

4.5 Online personalization and safe assistance

Use calibration data for initial open/closed pinch range, typical reach speed and baseline jitter. After each successful practice attempt, update user-specific parameters by a bounded exponentially weighted moving average (EWMA). Assistance should be **modest, transparent and reversible** - e.g., enlarge a grasp zone by at most a configured small fraction or damp noisy pointer motion, rather than silently moving objects for the user.

```
if valid_trial and adaptation_enabled:
    profile.jitter = clamp(0.85*profile.jitter + 0.15*trial_jitter, limits)
    profile.grasp_tolerance = clamp(base_tolerance + k*profile.jitter,
                                    min_tolerance, max_tolerance)
assist_gain = clamp(1 - predicted_uncertainty / uncertainty_limit, 0, 1)
if tracking_lost or predicted_uncertainty_high: assist_gain = 0
```

No overnight deep learning required

MediaPipe provides pretrained hand landmarks; Kalman filtering and a small classifier use light computation; personalization is online statistical adaptation. Do not present deterministic physics formulas or a rule-based controller as a newly trained neural network.

5. Simulation: virtual hand, objects and grasp physics

5.1 Scope and implementation choice

Start with a **2D Pygame environment**. Draw two fingertip markers or small opposing virtual gripper pads that follow the tracked thumb-index geometry, plus a reach cursor. A detailed 3D hand mesh is optional polish only. Keep simulation equations and AI policies independent of rendering so the simulator can be tested headlessly.

5.2 World coordinate system and fixed-step loop

- Use an internal 960x540 or 1280x720 coordinate canvas and map camera coordinates by a calibrated affine transform.
- World units are initially pixels and normalized contact coefficients. Do not call computed values newtons unless an actual force calibration is performed.
- Use a fixed simulation time step (e.g., 1/60 s), limit total catch-up steps after pauses, and interpolate rendering if necessary.
- Each object stores ID, type, position, collision box/radius, virtual mass, stiffness, friction factor, damage threshold and state.
- Maintain separate actual pointer, predicted pointer, contact pads, object center, assistance target and feedback trigger state.

5.3 Three object profiles and outcomes

Virtual object	Contact behavior	Success or failure
Fragile glass	Tight grip tolerance, low simulated damage threshold	Holding works in a narrow effective-grip interval; high closure produces virtual damage.
Sponge	Low stiffness, gradual virtual deformation	More closing is allowed; visual compression increases with q.
Heavy block	Requires sustained high simulated holding level	Object slips if simulated holding is below required threshold.

For each object, define a **contact closure** q_contact and a virtual grip effort G = k_object * max(0, q - q_contact). A slip check compares G with a chosen threshold T_hold; a break check compares G with T_break (if fragile). These are educational model parameters, not a measurement of a human hand.

```
G = stiffness[obj] * max(0.0, closure_q - contact_threshold[obj])
if not geometric_overlap: state = "NO_CONTACT"
elif G < hold_threshold[obj]: state = "SLIPPING"
elif G > break_threshold[obj]: state = "DAMAGED"   # for fragile object
else: state = "STABLE_GRASP"
# On each state change, publish exactly one event to hardware policy.
```

5.4 More realistic physics without heavy simulation

If time permits, add a first-order spring-damper effect for compressible objects and a gravitational slip animation for insufficient grasp. In a conceptual two-contact friction model, the required normal force per contact is approximately m*g/(2*mu) for symmetric frictional holding; without calibrated force sensing, use the formula only to shape relative simulated parameters, not to report real forces.

- **Visual feedback:** use consistent cues for contact, slipping, successful hold and simulated break; show the AI-estimated grasp state and uncertainty.
- **Assistive mechanism:** optionally snap only within a bounded radius or stabilize jitter; a full automatic grasp is prohibited in assisted-vs-unassisted comparisons.
- **Failure realism:** lower confidence or faster closure can cause false contact and should be displayed rather than hidden.
- **Replay:** record timestamped pointer/closure events and replay without the webcam, preserving trial reproducibility.

5.5 Simulation test cases

Case	Input	Expected result
T01 - open hand	q close to 0; no contact	No virtual grasp and no active servo cue.
T02 - valid fragile grip	Overlap + q within valid band	Stable grasp event; optional bounded contact cue.
T03 - fragile over-grip	High q on fragile object	Break animation + no repeated servo chatter.
T04 - heavy under-grip	Low q on heavy object	Slip/fall animation.
T05 - lost tracking	No hand landmarks for timeout	Virtual safe release / pause; actuator neutral.
T06 - low confidence	Pose inconsistent or occluded	No active physical cue; show unreliable state.
T07 - repeated edge crossing	Jitter near collision boundary	Hysteresis prevents repeated contact events.

6. Haptic mechanism: what it can and cannot do

6.1 Definition: actuator, sensor, perceived sensation

The Grove piezo module is a **digital vibration sensor**, not a haptic vibration actuator. The Grove touch module detects capacitive proximity/touch; it does not measure pressure. The ±1.5g accelerometer measures acceleration; it cannot report applied fingertip force or human perception. Your only confirmed movement actuator for this prototype is the separate servo. [R3-R5]

Question	Can we measure it with confirmed hardware?	Approach
What servo angle did software request?	YES	Log command angle, event and timestamp.
Did the fixture accelerate?	YES, approximately	Mount accelerometer on fixture; sample x/y/z within ±1.5g.
Was a vibration event detected?	YES, binary detection	Mount piezo appropriately; log digital HIGH/LOW transitions.
Did a user touch the pad?	YES, proximity-based	Grove touch sensor output; beware false positives near the pad.
What angle did the servo actually reach?	NOT directly	Servo commands do not prove achieved position; sensor proxy only.
What force did the fingertip feel?	NO	Needs load cell/FSR + calibration; absent from inventory.
Was the cue noticed / useful?	ONLY through user report + task metrics	Ask recognition question and compare assisted/unassisted trials.

6.2 Safe, simple contact fixture

Recommended initial build: fix the servo rigidly to a cardboard/wooden frame; attach a short, soft, padded pointer to its horn; keep a separate touch pad and finger rest so the finger cannot be pinched between moving parts. Ensure an unobstructed escape path. Add a physical stop on the control interface, start in non-contact mode and gradually inspect allowable travel.

- **Mode A - non-contact (required first):** servo moves a visible pointer next to the finger; buzzer and LEDs explain cue type. This verifies the control loop with no contact risk.
- **Mode B - guarded tactile cue (optional):** soft, short travel lightly taps a fingertip resting on an open foam surface after mechanical and power checks. No forced grip or rigid clamp.
- **Mode C - true vibrotactile device (future):** would require an actual vibration actuator and driver; the Grove piezo detector is not a substitute.

6.3 Haptic event policy and mapping

Virtual event	Physical output	Gate / limit
CONTACT	Single small-angle servo motion; optional green LED pulse	Only if confidence high, hardware armed and cue budget allows.
STABLE_HOLD	Green LED, optional LCD status; avoid continuous servo pressure	No continuous force against user.
SLIP	Blue LED and optional short buzzer; servo returns toward neutral	No repeated cue on every simulation frame.
DAMAGE	Red LED + one distinct buzzer pattern	Servo does not push harder to imitate damage.
TRACKING_LOST / STOP	Return servo to neutral if safe and stop new cues	Timeout and emergency stop take priority.
CALIBRATION	Display or LED status; no active tactile cues	User is not touching moving arm during calibration.

A hobby servo is not suitable for generating high-frequency textured vibrations. It can produce **discrete positional or tapping cues**. The exact travel angles and motion speeds must be chosen after identifying the servo model; this document intentionally does not prescribe a force or travel range as universally safe.

6.4 Measuring haptic performance

- **Command latency:** host timestamp at event -> Arduino command ACK; this is communication round-trip/proxy, not physical movement completion.
- **Physical response latency:** time from Arduino command receipt to first threshold-crossing measured by accelerometer or piezo. Use Arduino uptime stamps for both when possible.
- **Response consistency:** proportion of valid contact events followed by at least one sensor response within an agreed window.
- **Cue detection:** user indicates whether a cue occurred. Report hit rate, misses and false reports over randomized cue-present/cue-absent trials.
- **Task effectiveness:** success rate, completion time, over-grip/slip errors, and false assistance activations across experimental modes.

- **Calibration caveat:** acceleration and piezo triggers are not contact-force or tactile-intensity values.

6.5 Safety and power checklist

- Read servo label and specification before applying power; determine whether standard hobby servo or continuous-rotation device.
- Verify rated supply voltage and current including stall demand; use a regulated supply rated for the servo as necessary. Never wire a higher-voltage barrel adapter directly to the servo.
- Connect servo signal to a suitable Arduino digital pin; connect supply ground and Arduino ground together if using external servo power.
- Avoid loading Arduino 5V rail with a motor that causes resets or overheats the board; test first with low load.
- Physically limit travel and keep contacts padded, low force, and outside pinch points. Do not secure the servo to a finger.
- Provide immediate stop control: hardware button -> disarm; software stop -> disarm; watchdog timeout -> neutral/disabled behavior.
- If servo is hot, chatters continuously, causes brownouts or accelerometer saturates, stop testing and revert to non-contact mode.

Safety gate - no human contact until passed

Servo model identified; supply measured and safe; travel and fixture tested while unloaded; finger can move away freely; emergency stop works; neutral state and watchdog tested; no sharp/hot parts; a team member observes the first trial.

7. Pin mapping, circuit plan and Arduino firmware

7.1 Proposed wiring map (subject to physical pin access)

This uses the Grove Base Shield V2 on an Arduino Uno. The shield supports Grove digital, analog and I2C connectors; set its power switch correctly for the Uno as instructed by Seeed [R2]. The exact servo lead configuration depends on the actual motor and whether the shield leaves its D9 header accessible.

Component	Tentative connection	Note
Grove Touch Sensor	D2 Grove digital port	Arm/touch input; proximity is not force.
Grove Button	D3 Grove digital port	Explicit disarm / calibration button; debounced.
Grove Green LED	D4 Grove digital port	Stable grasp / heartbeat.
Grove Red LED	D5 Grove digital port	Error / unsafe state.
Grove Piezo Vibration Sensor	D6 Grove digital port	Binary event; threshold calibrate if adjustable.
Grove Buzzer (optional)	D7 Grove digital port	Short low-volume cue.
Grove Rotary Angle Sensor(P)	A0 Grove analog port	User-selected assistance / intensity parameter.

Component	Tentative connection	Note
Grove 3-axis accelerometer	I2C Grove port	MMA7660FC; address 0x4C according to Seeed [R3].
Grove RGB LCD (optional)	I2C Grove port	Older 0x3E text / 0x62 RGB; newer RGB 0x30. Check version [R6].
Servo signal	D9 header / adapter lead	Servo PWM via Servo library; power separately if required.
Arduino Uno -> laptop	USB serial	Single host process owns serial port.

The Grove kit contains several compatible ports but shared I2C devices require correct addresses. If shields/headers prevent accessing D9 or if pin conflicts arise, revise the allocation and keep it in a single **pins.h** file. Do not assume every Grove module communicates through the same electrical interface.

7.2 Simplified electrical topology

```
Laptop USB ----> Arduino Uno + Grove Base Shield V2
                       |-- D2  Touch sensor
                       |-- D3  Stop / calibration button
                       |-- D4  Green LED
                       |-- D5  Red LED
                       |-- D6  Piezo vibration event
                       |-- D7  Buzzer (optional)
                       |-- A0  Rotary angle sensor
                       |-- I2C Accelerometer (0x4C) / LCD (verify revision)
                       +-- D9  Servo SIGNAL ONLY
Regulated servo supply (+) ---> Servo V+  [only when required by model]
Regulated servo supply (-) ---> Servo GND -- shared with Arduino GND
```

7.3 Arduino responsibilities

- Initialize pins, I2C and serial without moving the servo suddenly; start DISARMED and safe.
- Parse bounded newline-terminated messages; reject unexpected commands, values or stale sequence numbers.
- Apply angle limits, minimum time between cues, neutral-return and watchdog behavior.
- Read touch/button, accelerometer and piezo; send compact sensor packets with Arduino uptime.
- Acknowledge commands and expose current arming/status state; log rejects so the host can show them.
- Continue safe behavior even if Python crashes or serial cable disconnects.

7.4 Suggested serial protocol v1

Use ASCII newline-delimited records at 115200 baud for simplicity. Example fields are illustrative and should be validated; use a monotonic sequence number, a fixed header and explicit state. Avoid unrestricted text execution on the controller.

```
Host -> UNO:   ARM,seq
Host -> UNO:   CUE,seq,event,angle_deg,duration_ms
Host -> UNO:   STOP,seq
UNO  -> Host:  ACK,seq,uptime_ms,status
UNO  -> Host:  S,uptime_ms,ax_g,ay_g,az_g,piezo,touch,pot,armed
UNO  -> Host:  ERR,seq,reason

Example: CUE,104,CONTACT,82,90
Example: ACK,104,172802,OK
```

Important: host monotonic clock timestamps and Arduino millis() timestamps are from different clocks. Do not subtract them directly. For physical response delay use Arduino-side timestamps, or perform an explicit clock-offset estimation / round-trip method before comparing host and device timestamps.

7.5 Safety state machine

Firmware states: BOOT -> DISARMED -> ARMED_IDLE -> CUE_ACTIVE -> ARMED_IDLE; any state -> FAULT/DISARMED. Enter FAULT on malformed repeated packets, unacceptable supply symptoms, stop button or watchdog expiry. Require an explicit re-arm after an emergency stop. The user-facing controller may always select non-contact mode regardless of firmware state.

8. Software stack and repository structure

8.1 Preferred hackathon stack

Layer	Recommended tool	Justification
Language and environment	Python 3.11 virtual environment	Widely compatible ecosystem; lock tested versions after setup.
Camera capture	opencv-python	Direct webcam frames, preview and frame timestamps.
Hand model	MediaPipe Tasks Hand Landmarker	Pretrained one-hand 21-landmark perception; requires model asset [R1].
Feature engineering	NumPy / optional SciPy	Distances, smoothing, temporal windows and filtering.
Prediction	Custom constant-velocity Kalman filter or FilterPy	Lightweight temporal state estimation.
Grasp classification	Rules initially; scikit-learn optional	Fast, interpretable baseline with optional trained improvement.
Virtual simulation	Pygame	Low-friction 2D rendering, simple update loop, easy live demo.
Arduino firmware	Arduino IDE + Arduino Servo + Grove libraries	Physical IO and safe actuator control.
USB communication	PySerial	Simple cross-platform host-to-board serial contract.
Logging and metrics	CSV, pandas, matplotlib	Reproducible trial logs and baseline comparisons.
Tests	pytest	Deterministic unit tests for equations, messages and state transitions.

Avoid a mandatory FastAPI + React + WebSockets architecture for the MVP. It adds real-time synchronization, deployment and UI overhead without strengthening the main research claim. Add a separate dashboard only after the demo loop is complete.

8.2 Suggested repo layout

```
ghostgrip/
  README.md                # quick start, consent and hardware photos
  requirements.txt         # frozen tested versions
  config.yaml              # thresholds, pins, simulator and servo limits
  assets/hand_landmarker.task
  src/
    app.py                  # launcher and mode selection
    vision/camera.py        # OpenCV + MediaPipe Tasks
    ai/features.py          # pinch/velocity/quality
    ai/intent.py            # rule-based + optional classifier
    ai/predict.py           # Kalman filter and prediction buffer
    ai/adapt.py             # calibration + EWMA profile
    simulation/world.py     # fixed step contact physics
    simulation/objects.py   # object profiles
    hardware/serial_link.py # packets, ACK, retries, watchdog
    hardware/haptics.py     # safety gating + cue scheduler
    metrics/logger.py       # CSV sessions and event timeline
    ui/pygame_app.py        # visual layer + status
  firmware/ghostgrip_uno/ghostgrip_uno.ino
  tests/                    # headless AI + physics + protocol tests
  data/synthetic/           # fake demo traces (safe to commit)
  data/local/               # ignore real volunteer recordings
  docs/                     # pin map, risk register, demo script
```

8.3 Environment bootstrap and dependencies

```
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# macOS/Linux:
source .venv/bin/activate

python -m pip install --upgrade pip
pip install mediapipe opencv-python numpy pygame pyserial pandas matplotlib pytest
# Optional after core MVP: pip install scikit-learn filterpy pyyaml
pip freeze > requirements.txt
```

MediaPipe Tasks requires a compatible .task hand-landmarker model asset. Download it from the official Google AI documentation and keep its path in configuration. Verify that the laptop actually opens the webcam and that MediaPipe can process a frame **before** starting the physics work. Software installation details can change; pin versions only after testing on the hackathon machine.

8.4 Configuration file: single source of truth

```
camera: {index: 0, mirror: true, target_fps: 30}
simulation: {width: 960, height: 540, physics_hz: 60}
ai: {prediction_horizon_ms: 120, grip_confirm_frames: 3,
     adapt_alpha: 0.15, low_confidence_behavior: "disable_assist"}
hardware: {enabled: true, port: "AUTO", baud: 115200,
           cue_cooldown_ms: 500, watchdog_ms: 500,
           servo_neutral_deg: "VERIFY", servo_min_deg: "VERIFY",
           servo_max_deg: "VERIFY", contact_mode: false}
experiment: {save_raw_video: false, participant_id: "anonymous"}
```

Do not send configuration values such as the placeholder "VERIFY" to Arduino. The program must refuse to arm until a verified servo range is supplied. The shown YAML is a design example; validate types when implementing the loader.

9. End-to-end interactions and user experience

9.1 Launch and calibration sequence

- **Start screen:** check webcam, model file and Arduino presence; show laptop-only fallback if hardware missing.
- **Consent screen:** explain that this is a prototype, not medical advice; state what data will be saved.
- **Hand calibration:** show open-hand and pinch prompts; estimate per-user d_open/d_closed and pointer range.
- **Optional physical calibration:** with motor disarmed, confirm touch pad, LEDs and accelerometer; set safe non-contact servo range.
- **Ready check:** verify no tracking loss, at least three reliable calibration samples and no active hardware fault.
- **Mode selection:** Baseline / AI-only / Haptics-only / AI+haptics; display current condition clearly.

9.2 A complete trial

The user moves a tracked hand toward a virtual object, closes thumb and index finger, attempts to hold the object for a short fixed dwell, transports it to a target zone if desired, and releases. The simulator evaluates overlap, closure, simulated effort, stability and timing. A bounded physical event can be triggered on first valid contact when the haptic condition is enabled.

Moment	Visible screen behavior	AI / hardware behavior
Reach	Object boundary; actual pointer; optional forecast ghost	Track landmarks, update velocity, predict future pointer.
Grasp attempt	Grip meter increases; object contact zone highlights	Estimate grasp phase, check overlap and thresholds.
Contact	One contact animation and event annotation	Validate confidence; publish at most one safe servo cue.
Hold / failure	Object carried, slips, deforms or breaks	Apply simulation state and log error reason.
Release	Object drops to target / returns; success overlay	Record completion time; optionally update adaptive profile.
After trial	Compact result summary and next trial button	Write CSV and transition hardware to neutral.

9.3 Screens and overlays

- **Main viewport:** one virtual object, hand cursor, ghost prediction marker, target and contact status.
- **Metrics strip:** tracker quality, grip closure, predicted 120-ms endpoint, AI mode, haptic armed/disarmed state, object state.
- **Technician panel:** serial port, ACK rate, last hardware event, accelerometer values, piezo event and emergency stop.
- **Experiment screen:** anonymous participant ID, condition order, trial count, completion time and saved metrics.
- **Presenter screen:** toggle baseline vs AI, optionally show trajectory and model uncertainty without exposing personal data.

10. Logging, measurements and evaluation strategy

10.1 Four experimental conditions

Condition	AI assistance	Physical cue	Purpose
A: Baseline	OFF	OFF	Unassisted reference; rules still define simulated contact.
B: AI only	ON	OFF	Estimate contribution of prediction and personalization.
C: Haptic only	OFF	ON	Estimate contribution of physical contact cue.
D: AI + Haptic	ON	ON	Evaluate combined system and interactions.

Within a session, hold object layout and task instructions constant and randomize or counterbalance the order of conditions. Calibration stays fixed for a comparison block; adaptive updates should be frozen while measuring a condition unless adaptation itself is the variable under study.

10.2 Primary metrics with precise definitions

Metric	Calculation	Reporting caution
Grasp success rate	successful trials / total valid trials * 100%	Task-based simulated criterion, not medical recovery.
Completion time	time of successful release - trial start	Report median and range; note timeout failures.
Slip rate	trials with >=1 simulated slip / valid trials	Simulation model outcome, not real object slip.
Over-grip rate	trials exceeding virtual break threshold / valid trials	The threshold is synthetic unless calibrated.
Prediction error	mean ||p_pred(t+h) - p_obs(t+h)||	Measured in normalized coordinates or pixels.
Intent F1 / confusion matrix	classified vs manually annotated action-window labels	Tiny data may not generalize to other users.
Contact-to-ACK delay	host send to host ACK, corrected for interpretation	Network/serial round-trip proxy, not tactile onset.
Physical cue detected	piezo/accel response following a logged command	Sensor responsiveness, not felt intensity.
Subjective cue recognition	correct user reported present/absent / probes	Small, unblinded testing can be biased.

10.3 Event and sample schema

```
session_id, trial_id, condition, host_time_monotonic_ms, event,
object_id, object_type, hand_quality, cursor_x, cursor_y,
closure_q, intent_state, future_x, future_y, prediction_sigma,
sim_effort, sim_outcome, assistance_gain, cue_seq,
servo_angle_commanded_deg, ack_host_ms, arduino_uptime_ms,
accel_x_g, accel_y_g, accel_z_g, piezo_event, touch_event
```

Use separate tables for frame-level samples, device-level samples and trial-level outcomes to avoid duplicating large numbers of fields and to keep timestamps unambiguous. A lightweight CSV + pandas workflow is enough; no database is necessary for the hackathon.

10.4 Experimental procedure (small feasibility study)

- Aim for 3-5 consenting adult volunteers if available, with 3 object types and a small number of repeated attempts per condition.
- Randomize mode order; record whether each volunteer practiced before testing; give the same short practice block.
- Use a recorded standardized set of virtual object properties and clearly mark each trial state.
- Predefine what counts as valid trial and dropout; do not hide failures or selectively delete slow runs.
- Show per-participant results and descriptive aggregate results; avoid significance or clinical efficacy claims with tiny sample sizes.

- Review raw traces for unexpected AI assistance and accidental servo commands before presenting aggregate numbers.

10.5 Success thresholds and demo scorecard

Gate	Target (engineering aspiration)	If missed
Visual tracking	Stable hand and cursor for 20 seconds in demo lighting	Move camera, improve light, simplify scene.
Baseline physics	3 object types produce distinct outcomes reproducibly	Remove extra effects; use state machine and thresholds.
AI method	Prediction error logged; baseline vs predictor comparison available	Use simpler constant-velocity filter; never fake improvement.
Arduino link	Commands and ACK observable in app, with disarm working	Show non-contact LED-only mode.
Haptic event	At least one repeatable sensor-detected or visually observed servo response	Label unmeasured physical response as unavailable.
Results	At least one honest A/B trial table, including raw count	Use qualitative demo and state that sample is preliminary.

11. Implementation plan: 36-hour hackathon

36-HOUR CRITICAL PATH



Time	Main output	Definition of done / fallback
0-4 h	Hardware inventory; project skeleton; camera and serial hello-world	Hand points visible. Servo model checked. If not, non-contact LED proof.
4-10 h	Hand feature pipeline and calibrated virtual pointer	Thumb-index closure and cursor stable; logging works.
10-17 h	3-object 2D simulator and deterministic grasp baseline	Glass/sponge/block states work in tests and can be demoed.
17-24 h	Kalman predictor, uncertainty, adaptive tolerance, experiment modes	AI versus baseline switch and metrics display; disable ineffective prediction.
24-30 h	Servo safety control, touch/LED + optional acceleration/piezo readback	One reliable event -> bounded cue; disarm, timeouts, no pinches.
30-36 h	Test sessions, data visualization, pitch, repository and backup demo	One complete 90-s walkthrough, measured metrics and replay fallback.

11.1 Parallel workstreams for 4 people

Owner	Primary responsibility	Integration contract
A - Perception/ML	Camera, MediaPipe, features, predictor, personalization	Publishes timestamped HandState with position, q, confidence.
B - Simulation/UI	Pygame world, objects, control modes, animations	Consumes HandState, emits typed SimEvent and TrialOutcome.
C - Embedded/haptics	Arduino, Grove shield, servo, safety, serial client	Consumes bounded CueEvent, returns DeviceState and ACK.
D - Integration/evaluation	Configuration, logs, tests, demo, metrics, pitch	Owns main loop, CI/smoke script and analysis notebook.

With three people, merge integration/evaluation into the simulation workstream and assign one person sole ownership of the hardware/safety design. Integrate every 3-4 hours; do not allow three branches to evolve for 25 hours without an end-to-end test.

11.2 Milestones and kill switches

- **Hour 4:** webcam and Arduino work independently. If hardware problems persist, continue laptop-only AI work.
- **Hour 10:** if hand tracking is unstable, use controlled lighting and a single central camera region; disable optional 3D styling.
- **Hour 17:** simulation must be complete before adding optional ML or extra sensors.
- **Hour 24:** if prediction is worse than baseline, use it only for visualization or disable assistance; show honest metrics.
- **Hour 30:** freeze features. Only safety fixes, bugs, recorded demo and presentation allowed.

12. Testing strategy and fault tolerance

12.1 Software unit tests

Subsystem	Example test	Expected
Geometry	Open and closed calibration, zero palm scale	q remains within [0,1]; no division by zero.
Temporal	Dropped frames, variable dt, out-of-order timestamps	No NaN, no negative dt, filter can reset cleanly.
Intent	Hand jitter near threshold and slow closure	Hysteresis prevents rapid transition spam.
Physics	Identical inputs at different display FPS	Same object outcome for fixed time-step simulation.
Adaptation	Extreme user jitter / misleading measurements	Parameters stay within configured safe bounds.
Protocol	Malformed packet, out-of-range angle, stale sequence	Arduino rejects; never moves from rejected command.
Watchdog	Disconnect serial during active feedback	Firmware disarms or returns toward neutral as designed.
Logging	Restart application halfway through session	Existing data stays readable; incomplete trial flagged.

12.2 Physical integration and safety tests

- Test LED and touch without servo. Then test servo disconnected from user and without fixture load.
- Verify servo voltage at connector with multimeter and inspect for Arduino resets.
- Confirm neutral and endpoint angles, then reduce working travel to the smallest reliable range.
- Observe 10 repeated cue cycles and confirm no overheating, gear strain, detachment or unusual noise.
- Move accelerometer by hand to confirm orientation and gravity offsets; do not treat constant 1g as actuator force.
- Trigger piezo sensor manually to verify threshold detection; check false triggers from table vibration.
- Press emergency stop mid-cue; power-cycle and verify it does not re-arm automatically.
- Run a 5-minute continuous software soak test with no user contact and log loss/errors.

12.3 Failure modes and responses

Risk	Severity	Detection	Fallback
Servo pinches or moves unexpectedly	HIGH	Unsafe travel, mechanical interference, emergency stop	Non-contact display-only mode; no finger contact.
Power brownout or unstable servo	HIGH	Uno resets, hot wires, noisy/halted servo	Disconnect servo; regulated compatible supply or LEDs only.
Camera loses hand / occlusion	MED	Tracking quality bad for several frames	Pause trial, neutral physical response, request recalibration.
AI predicts wrong target	MED	Prediction error/uncertainty high	Cap assistance; switch to observed-position baseline.
Serial connection fails	MED	No ACK / watchdog timeout	Hardware disarms, virtual simulation continues.
Sensors miss tactile event	LOW	No accelerometer/piezo threshold crossed	Label measurement missing; do not infer zero sensation.
Presentation time runs out	MED	Demo exceeds planned 90 s	Use pre-scripted 3-object demo + recording.

13. Demonstration, judging story and technical evidence

13.1 Ninety-second live demo

Time	Narration and action
0-10 s	"A webcam can see a hand, but it cannot tell whether a difficult grasp is stable. GhostGrip combines prediction, simulation and feedback."
10-25 s	Show calibrated hand and virtual object. Baseline grasp attempts cause a slip or overshoot.
25-42 s	Enable adaptive AI. Show predicted pointer and a tracked uncertainty value; repeat the same object challenge.
42-58 s	Enable guarded feedback. The first real contact triggers one servo/LED cue; show sensor/ACK on screen.
58-75 s	Swap fragile glass for sponge/heavy block and demonstrate different contact outcomes with the same hand.
75-90 s	Display actual recorded baseline vs AI metrics and conclude with clear limitations and next steps.

13.2 Judges can challenge the system

- Ask a judge to move the hand partially out of frame: the prediction/servo should disable safely.
- Ask them to switch object types without restarting the application: contact physics should change immediately.
- Let a judge compare baseline, AI and haptic modes to distinguish what each system adds.
- Ask them to press the stop button: the device should stop active feedback while the laptop still runs.
- If sensor feedback is connected, display raw acceleration or piezo event alongside every servo command.

13.3 Results slide / poster structure

- **The user problem:** lack of adaptive, tangible feedback in low-cost virtual grasp interactions.
- **Innovation:** pretrained perception + temporal intention estimate + uncertainty-aware personalization + controllable physical cues.
- **Architecture:** camera -> feature engineering -> predictor -> physics -> safe controller -> Arduino.
- **Evidence:** video throughput, prediction MAE, simulation outcome, task success and cue-response logging; use actual measured numbers only.
- **Known limits:** simulation surrogate instead of real finger force; sample size too small for clinical or general usability claims.
- **Next iteration:** affordable force sensor or proper haptic actuator, 3D simulation, better user study.

14. Deployment, operations, privacy and reproducibility

14.1 Local deployment

Run GhostGrip locally. The laptop handles camera data and Python inference; Arduino uses a USB connection. The design works without internet after dependencies and model asset are installed. Avoid cloud calls in the live demo to reduce network and privacy risks.

- Pre-install Python packages, model file, Arduino firmware and USB-serial drivers.
- Start the Uno, then run a preflight script to detect camera, model, serial port and optional Grove sensors.
- Launch app in baseline/servo-disarmed mode; configure safe servo range using confirmed model.
- Perform calibration, then switch to trial/experiment mode.
- At end, export anonymized metrics and reset the physical output; do not auto-enable servo on restart.

14.2 Data privacy and consent

Hand movement can be personal behavioral data. Use an anonymous session ID, process raw video locally, and avoid storing or uploading camera footage by default. Save only event features and summary statistics needed for the demo; if video recording is necessary, get explicit consent, use an opt-out path and delete footage after the event according to team policy.

14.3 Build reproducibility checklist

- Document exact servo label and power supply photos, shield pin configuration and wiring version.
- Freeze requirements.txt after the project runs correctly; record OS/Python/Arduino IDE versions.
- Provide example config.yaml with no secrets or machine-specific absolute paths.
- Document model asset source and license conditions; do not commit large video datasets without permission.
- Include one command for laptop-only replay and one for normal hardware-enabled startup.
- Provide unit tests, fixture safety photos, architecture diagram, concise calibration instructions and known bugs.

15. Decision log and open engineering questions

Decision	Current choice	Reason / what to verify
AI runtime	Local Python on laptop	Avoid trying to run MediaPipe or ML models on Arduino Uno.
Simulation	2D Pygame with three object profiles	Small enough for reliable end-to-end demo.
AI baseline	Rule-based grasp state + temporal filter	Always available; optional classifier evaluated against it.
Haptic device	Servo positional cue; non-contact first	Only confirmed actuator; not realistic vibration texture.
Sensing	Touch + accel/piezo if feasible	Measures events/motion, not contact force.
Servo model	OPEN	Must inspect label, rated voltage/current, control mode.
Power	OPEN	Confirm source and regulated servo supply before arming.
Mount / padding	OPEN	Need stable fixture and safe touch-point design.
Software target OS	OPEN	Verify installed camera/serial drivers and package support.
Team roles	OPEN	Assign module ownership and integration lead.
Experimental volunteers	OPEN	Confirm consent and demonstration-only use.
Final success claims	NOT YET MEASURED	Do not write invented model accuracy, force or improvement percentages.

Go/no-go rule

If a low-force, pinch-free tactile fixture cannot be made safe with confirmed materials, still complete GhostGrip as an AI-first interactive simulation with **instrumented, non-contact servo feedback**. This is a legitimate prototype and is better than an unsafe or nonfunctional glove.

16. Implementation pseudocode and integration contract

16.1 Main app loop

```
init_camera(); init_hand_model(); init_world(); init_logging()
link = connect_arduino_or_mock(); link.disarm()
profile = calibrate_hand_when_user_ready()
while app_running:
    frame, host_ts = camera.read_timestamped()
    result = hand_landmarker.detect_for_video(frame_to_mp(frame), host_ts)
    hand = features(result, profile, dt_from_clock())
    predicted = predictor.update_and_forecast(hand)
    intent = state_machine.update(hand, predicted)
    events, outcome = world.step_fixed(hand, intent, condition)
    for event in events:
        command = haptic_policy.filter(event, hand.quality, link.state)
        if command is not None: link.send(command)
    link.poll_sensor_packets()
    logger.append(hand, predicted, intent, events, outcome, link.state)
    ui.render(world, hand, predicted, link, condition)
link.stop_and_disarm(); logger.close()
```

16.2 Typed data contracts between workstreams

```
HandState(t_ms, pointer_xy, closure_q, dq_dt, velocity_xy,
          stable_score, tracking_ok, tracking_quality)
Prediction(t_query_ms, horizon_ms, future_pointer_xy, sigma_xy)
SimEvent(event_id, t_ms, object_id, kind, simulated_grip_effort)
CueEvent(seq, kind, safe_angle_deg, duration_ms, no_contact_mode)
DeviceState(uptime_ms, armed, touch, piezo, ax_g, ay_g, az_g,
            rotary_raw, last_ack_seq)
TrialOutcome(session_id, trial_id, condition, success, fail_reason,
             start_ms, end_ms, slip_count, overgrip_count)
```

16.3 Example integration questions to resolve early

- Does the simulator receive landmarks even when camera confidence is low, or a typed TRACKING_LOST event? Decide one format.
- Who defines the camera-to-world mirror transform? Have one owner and unit tests.
- Is grip q normalized by individual calibration and is its direction consistent with all object properties?
- Does the hardware controller receive one event per contact transition or every simulation frame? It must be one event.

- Who owns the clock for timestamps and how are Arduino readings aligned to host events?
- When a trial ends, is the adaptive profile immediately updated? Make this explicit to prevent biased comparisons.

17. Suggested enhancements only after MVP passes

Enhancement	Benefit	Why defer it
Small trainable intent classifier	Potentially improves motion phase recognition	Requires careful labels and evaluation; rules may be sufficient.
3D hand + object environment	More compelling visual storytelling	3D geometry and occlusion add substantial complexity.
Real force sensing (FSR/load cell)	Actual physical normal force estimate after calibration	Parts are not present in confirmed hardware kit.
Vibration motor + driver	Real vibrotactile patterns and pulse-width control	Actuator, driver, power and mechanical design are additional.
Mechanically braked / compliant haptic interface	More realistic resistance feedback	Safety and control validation exceed hackathon budget.
Longitudinal personalized model	Adaptation across multiple sessions	Needs privacy controls and meaningful repeated data.
Remote teleoperation mode	Prediction under simulated network latency	Useful extension; not essential to core grasp demo.

18. Reference documentation and evidence basis

The hardware inventory was transcribed from the team-provided photograph of the Grove starter kit box and the team confirmation of an Arduino Uno and servo. Earlier UB Hacking winners were used as inspiration for presentation quality, not as evidence that GhostGrip is previously validated or likely to win. External links below provide component/API verification; the project architecture and engineering choices are our proposed design.

R1. Google AI / MediaPipe Hand Landmarker Python API, video and live-stream modes
https://ai.google.dev/edge/api/mediapipe/python/mp/tasks/vision/HandLandmarker

R2. Seeed Studio, Grove Base Shield V2 hardware overview
https://wiki.seeedstudio.com/Base_Shield_V2/

R3. Seeed Studio, Grove 3-Axis Digital Accelerometer (+/-1.5g), MMA7660FC
https://wiki.seeedstudio.com/Grove-3-Axis_Digital_Accelerometer-1.5g/

R4. Seeed Studio, Grove Piezo Vibration Sensor (digital output)
https://wiki.seeedstudio.com/Grove-Piezo_Vibration_Sensor/

R5. Seeed Studio, Grove Touch Sensor (capacitive proximity)
https://wiki.seeedstudio.com/Grove-Touch_Sensor/

R6. Seeed Studio, Grove LCD RGB Backlight version/address notes
https://wiki.seeedstudio.com/Grove-LCD_RGB_Backlight/

R7. Seeed Studio, Grove Rotary Angle Sensor(P)
https://wiki.seeedstudio.com/Grove-Rotary_Angle_Sensor/

R8. Arduino, Servo Motor Basics (including power considerations)
https://docs.arduino.cc/learn/electronics/servo-motors/

R9. scikit-learn, Probability Calibration
https://scikit-learn.org/stable/modules/calibration.html

R10. Pygame documentation
https://www.pygame.org/docs/

R11. PySerial API
https://pyserial.readthedocs.io/en/latest/pyserial_api.html

18.1 Engineering assumptions and source boundaries

- The servo model, mounting hardware, power supply compatibility and camera hardware were not fully identified by the supplied pictures; they are explicit unknowns.
- The pictured Grove kit lists one accelerometer and one piezo vibration detector. Neither provides real calibrated grip-force measurements.
- The physics model is illustrative and uses synthetic material parameters; the proposed values are not measured engineering properties of glass, sponge or stone.
- Latency and accuracy targets are goals, not observed measurements; this documentation contains no fabricated experiment results.
- The demo is for research/educational interaction only and should not be described as treatment or a medical product.

19. Final team launch checklist

Ready?	Required proof before demo
[ ]	Webcam finds one hand; correct mirrored pointer and valid pinch calibration.
[ ]	Three virtual objects produce meaningfully different simulation outcomes.
[ ]	Rule baseline works; AI prediction and adaptation can be toggled independently.
[ ]	Prediction MAE and trial success are logged; no invented performance claims.
[ ]	Servo model/power confirmed; non-contact servo cue operates without resets.
[ ]	Button/stop, confidence gate, watchdog and safe neutral behavior tested.
[ ]	No finger/skin contact unless safe padded fixture has passed review.
[ ]	Sensor readbacks are labeled correctly: acceleration and binary vibration only.
[ ]	Baseline, AI-only, Haptic-only and Combined modes can be selected.
[ ]	Recorded fallback, clean README, wiring diagram and presentation prepared.

Project statement for the final presentation

GhostGrip is a low-cost, AI-powered adaptive grasp simulator that tracks the hand in real time, forecasts near-future movement, personalizes virtual contact assistance and turns grasp events into bounded physical feedback through Arduino. It demonstrates how perception, uncertainty-aware AI, simplified physics and embedded hardware can operate as one measurable human-computer interaction loop.

END OF ENGINEERING SPECIFICATION | V1.0 | Pending servo model and hardware bring-up